DNSHawk: Complete dataset, preprocessing, model training, validation, and testing methodology

Your project, “Incident Detection for a DNS Tunneling Attack,” uses a PCAP-based machine-learning pipeline to detect DNS tunneling from DNS query behavior. We started with a large collection of packet captures, extracted 60-second DNS-window features, trained baseline XGBoost models A/B/C, and then developed wildcard-aware models D1/D2/D3.

The most important distinction throughout the experiment is:

- Training set: used to fit model parameters.
- Validation set: used to evaluate models during development, using data not used to fit those models.
- Final test set: held-out traffic used to evaluate generalization.
- Wildcard training captures: benign wildcard DNS traffic deliberately added to the training data for D1/D2/D3.
- Reserved wildcard test captures: five separate captures held out to evaluate false positives on wildcard DNS.

I will walk through the complete pipeline in chronological order, including the actual counts, split rules, features, metrics, and the methodological limitations we need to preserve when documenting your results.

1. The original dataset

We used the public DNS-Tunnel-Datasets collection from:

- DNS-Tunnel-Datasets GitHub repository
- The collection was also discussed in the context of the GraphTunnel DNS tunneling detection research paper.

The archive was stored in Google Drive at:

`/content/drive/MyDrive/DNS_Hawk/DNS-Tunnel-Datasets-main.zip`

It was extracted under:

`/content/drive/MyDrive/DNS_Hawk/dataset/raw/DNS-Tunnel-Datasets-main`

1.1 Dataset categories

The dataset contains five categories with different purposes.

Category	PCAP count	Label	Purpose

`normal`	68	Benign (0)	Ordinary DNS traffic for training and validation, with some captures reserved for later evaluation
`tunnel`	20 selected PCAPs	Tunnel (1)	Known DNS tunneling traffic used for training and validation
`unkownTunnel`	15 selected PCAPs	Tunnel (1)	Unseen tunneling tools, reserved for testing generalization
`crossEndPoint`	6 selected PCAPs	Tunnel (1)	Cross-endpoint tunneling evaluation
`wildcard`	13 PCAPs	Benign (0)	Benign wildcard DNS

			traffic used to study false positives

The dataset uses the spelling `unkownTunnel` in its directory/category name. It refers to unknown or previously unseen tunneling tools.

The first two categories supplied the main development data. The remaining categories were intended to challenge the models beyond the familiar training traffic.

1.2 PCAP manifest and exclusions

We created a manifest at:

`/content/drive/MyDrive/DNS_Hawk/dataset/processed/pcap_manifest.csv`

The manifest records each capture's relative path, category, traffic label, binary label, intended role, capture group, and file size.

There were 122 selected PCAP entries in the manifest after excluding nine parent PCAPs when corresponding same-stem directories contained segmented PCAP files. This prevented processing both a parent capture and its segments, which could duplicate packet data.

The manifest was the source of truth for the subsequent feature-extraction process.

2. PCAP preprocessing and feature extraction

The raw inputs were packet captures, not precomputed machine-learning feature tables. We used TShark to read DNS traffic and then aggregated it into windows.

122 selected PCAPs

Raw DNS traffic

TShark parsing

Extract DNS query and packet fields

DNS event records

Timestamp, IPs, query/domain, lengths, query type and related fields

60-second aggregation windows

Grouped by source IP, domain and window start

19 extracted features per window

Later reduced to 17 query-only features for XGBoost

2.1 DNS event extraction

The project's DNS event structure contains fields such as:

- `timestamp`
- `src_ip`
- `dst_ip`
- `query`
- `domain`
- `subdomain`
- `qtype`
- `query_length`
- `packet_length`
- `response_length`
- `rcode`
- `is_nxdomain`

TShark parses the packet captures and produces DNS events from which the feature extractor computes statistics.

2.2 Aggregation

The aggregation logic uses a 60-second window, grouped by:

`(src_ip, domain, window_start)`

Consequently, each feature row represents a particular source IP's DNS activity for a domain during a window. It is not one row per packet and not one row per entire PCAP.

This is important because the model learns from window-level behavior, such as query frequency, subdomain characteristics, and timing patterns.

2.3 The 19 original features

The feature extractor generated these 19 features:

No.	Feature	What it represents
1	`query_count`	Number of DNS queries in the window

2	`unique_query_count`	Number of distinct queries
3	`unique_query_ratio`	Unique-query count divided by total query count
4	`queries_per_second`	Query count divided by 60 seconds
5	`avg_query_length`	Average query length
6	`max_query_length`	Maximum query length
7	`entropy`	Shannon character entropy of the query data
8	`avg_subdomain_length`	Average subdomain length
9	`max_subdomain_length`	Maximum subdomain length

10	`avg_subdomain_entropy`	Average subdomain character entropy
11	`max_subdomain_entropy`	Maximum subdomain character entropy
12	`unique_subdomain_count`	Number of distinct subdomains
13	`unique_subdomain_ratio`	Distinct subdomains divided by nonempty subdomain observations
14	`nxdomain_rate`	Fraction of queries marked as NXDOMAIN
15	`mean_interarrival`	Mean time between consecutive queries

16	`std_interarrival`	Standard deviation of interarrival times
17	`avg_packet_length`	Average packet length
18	`max_packet_length`	Maximum packet length
19	`avg_response_length`	Average response length when available

The entropy features measure character diversity. Timing features characterize query intervals, while unique-subdomain statistics help distinguish repetitive benign queries from highly variable tunneling-like patterns.

The subdomain parsing was relatively simple rather than public-suffix-aware. That is an implementation limitation to acknowledge when describing the feature extractor.

2.4 Why the final models used 17 features, not 19

The final XGBoost models used the following 17 query-oriented features:

```
FEATURES = [    "query_count",    "unique_query_count",
"unique_query_ratio",    "queries_per_second",    "avg_query_length",
"max_query_length",    "entropy",    "avg_subdomain_length",
"max_subdomain_length",    "avg_subdomain_entropy",
"max_subdomain_entropy",    "unique_subdomain_count",
"unique_subdomain_ratio",    "mean_interarrival",    "std_interarrival",
"avg_packet_length",    "max_packet_length",]
```

We excluded:

- `nxdomain_rate`

- `avg_response_length`

The reason was a reliability issue in the live-capture implementation: the parser's response handling was not dependable enough to trust these response-derived values consistently. In particular, response lengths were being set to zero in the live pipeline, and response codes were not reliably available.

Keeping these two fields could have made offline training depend on information the deployed extractor could not reproduce accurately. Using 17 features made the intended offline-to-live feature schema more consistent.

2.5 Feature extraction results

The extraction process produced 898,737 feature rows across the 122 selected PCAPs.

Category	Feature rows
`normal`	895,694
`tunnel`	1,165
`unkownTunnel`	817
`crossEndPoint`	425
`wildcard`	636
Total	898,737

The per-capture CSVs were stored at:

`/content/drive/MyDrive/DNS_Hawk/dataset/processed/per_capture/`

The extraction checkpoint was stored at:

`/content/drive/MyDrive/DNS_Hawk/training/checkpoints/feature_checkpoint.csv`

The feature CSV filename was derived from the first 12 characters of the SHA-1 hash of each PCAP's relative path.

The extracted data passed the checks recorded at the time: feature schemas and labels were present, the numeric features had no missing or infinite values, and basic feature constraints

were satisfied.

Important imbalance: normal traffic contributed approximately 895,694 of the 898,737 rows, while known tunneling contributed only 1,165. Training directly on the entire extracted table without addressing the imbalance would have been problematic. The later training workflow therefore used query-count eligibility filters and benign downsampling.

3. Query-count filtering and the reason for Models A, B, and C

Not every 60-second window contains enough DNS activity to provide useful evidence of tunneling. We experimented with three minimum query-count thresholds:

Model	Minimum `query_count`	Meaning
A	10	Use windows containing at least 10 queries
B	5	Include windows containing at least 5 queries
C	2	Include windows containing at least 2 queries

The query-count filter was applied to the eligible training, validation, and evaluation rows for the respective experiments.

The original extracted dataset had this query-count distribution:

Category	Total rows	Below 10 queries	At least 10 queries
`normal`	895,694	893,631	2,063
`tunnel`	1,165	9	1,156
`unkownTunnel`	817	56	761
`crossEndPoint`	425	342	83
`wildcard`	636	56	580

This illustrates the tradeoff. A threshold of 10 excludes many benign low-activity windows but also limits the system's ability to analyze sparse tunneling activity. Lowering the threshold allows more windows to be evaluated, but introduces additional benign traffic and can increase false positives.

The thresholds of 10, 5, and 2 were experimental choices, not proof that traffic below the threshold is safe. The current pipeline simply does not evaluate such windows under the corresponding model configuration.

4. How the original training/validation split was performed

Before developing D1/D2/D3, we established the data-splitting strategy for the baseline models.

4.1 The split was group-aware

We used:

`StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)`

The first split produced the training and validation partitions.

This choice addresses two issues:

- Stratification: tries to preserve the class proportions across the folds.
- Group isolation: keeps rows belonging to the same capture group on one side of the split rather than distributing related windows between training and validation.

The group IDs were assigned as follows:

- Normal PCAPs received unique group IDs based on their relative paths: `normal::<relative_path>`.
- Related segmented captures in the other categories retained a shared group based on their category and original capture group: `category::original_capture_group`.

This was designed to reduce the risk of evaluating the model on windows that are too closely related to its training data.

4.2 Which categories were used for baseline training and validation?

The baseline training and validation sets used only:

- `normal` — benign label `0`
- `tunnel` — tunneling label `1`

The `unkownTunnel` and `crossEndPoint` categories were reserved for out-of-distribution-style testing, while wildcard traffic was used later in the wildcard-aware experiments.

The same group assignments from Model A were reused to reconstruct the split for Model C, maintaining the intended capture-level separation.

4.3 Why benign downsampling was necessary

The normal category had hundreds of thousands of rows, whereas the known tunnel category had only around a thousand.

To avoid training on an overwhelmingly benign dataset, the training pipeline capped the number of benign windows at three times the number of tunneling windows.

This is a maximum 3:1 benign-to-tunnel ratio, not a statement about the true prevalence of attacks in a real network.

Crucially, the benign downsampling was applied to the training data, not to the validation data. The validation sets remained unbalanced and therefore better reflected the eligible windows produced by the capture split.

The three baseline models used the same XGBoost configuration:

```
XGBClassifier(    objective="binary:logistic",    eval_metric="logloss",
n_estimators=400,    max_depth=5,    learning_rate=0.05,    subsample=0.8,
colsample_bytree=0.8,    min_child_weight=3,    reg_lambda=1.0,
random_state=42,    n_jobs=-1,    tree_method="hist",    device="cuda"  # or
"cpu")
```

The CPU version was preferred for final inference compatibility.

5. Baseline models A, B, and C

These models established the effect of the query-count threshold before adding wildcard examples.

All three used the same 17 query-only features and the same general XGBoost configuration.

Model A — minimum 10 queries

Eligible training data was filtered to windows with `query_count >= 10`, and benign training rows were sampled to the maximum 3:1 ratio.

Partition	Benign	Tunnel	Total
Training	1,609	1,070	2,679
Validation	454	86	540

Validation results at probability threshold `0.5`:

Metric	Result
Accuracy	99.63%
Precision	97.73%
Recall	100%
F1-score	98.85%
ROC-AUC	100%
Average precision	100%
True negatives	452
False positives	2
False negatives	0
True positives	86

Artifacts were saved under `/content/drive/MyDrive/DNS_Hawk/modelA/`, including the trained model, train/validation datasets, metadata, feature importance, and validation reports.

Model B — minimum 5 queries

Model B lowered the minimum query count to 5 and used the corresponding group-based split and training strategy.

Partition	Benign	Tunnel	Total
Training	2,981	1,076	4,057
Validation	880	87	967

Validation results at threshold `0.5`:

Metric	Result
Accuracy	99.90%
Precision	100%
Recall	98.85%
F1-score	99.42%
ROC-AUC	99.996%
Average precision	99.962%
True negatives	880
False positives	0
False negatives	1
True positives	86

Artifacts were saved under `/content/drive/MyDrive/DNS_Hawk/modelB/`.

Model C — minimum 2 queries

Model C lowered the threshold again, to `query_count >= 2`.

Its training partition initially contained 47,078 eligible rows, including 46,001 benign rows and 1,077 tunneling rows. The validation partition contained 13,758 rows, including 13,671 benign and 87 tunneling rows.

After training-set benign downsampling:

Partition	Benign	Tunnel	Total
Training after sampling	3,231	1,077	4,308
Validation, unsampled	13,671	87	13,758

The training set retained all eligible tunnel windows and sampled benign windows to maintain the maximum 3:1 ratio. About 42,770 eligible benign training windows were not used in that sampling draw.

Validation results at threshold `0.5`:

Metric	Result
Accuracy	99.88%
Precision	85.15%
Recall	98.85%
F1-score	91.49%
ROC-AUC	99.998%
Average precision	99.72%

True negatives	13,656
False positives	15
False negatives	1
True positives	86

The lower precision compared with Models A and B is consistent with Model C making more false-positive errors at its selected threshold, even though recall remained high.

Artifacts were saved under `/content/drive/MyDrive/DNS_Hawk/modelC/`.

5.1 Why these validation results were not sufficient

All three baseline models looked excellent on their validation sets. But validation performance alone did not demonstrate generalization to unseen tunneling tools or wildcard DNS.

We therefore constructed a held-out evaluation set using the unknown-tunnel, cross-endpoint, and wildcard categories. The early evaluation exposed a major problem: the baseline models classified a very large proportion of wildcard traffic as tunneling.

For example, on the earlier shared held-out set of 1,842 windows with `query_count >= 2`, the baseline models produced these results:

Model	**Accuracy**	**Recall**	**False positives on benign wildcard windows**
A	65.36%	95.81%	587
B	65.20%	94.24%	571
C	64.93%	94.82%	583

That was the motivation for the next phase: teach the models what benign wildcard DNS looks like without using the reserved wildcard test captures.

6. How the wildcard dataset was split for D1, D2, and D3

The wildcard category contained 13 PCAPs. We split these by capture, not by individual feature rows.

The assignment was saved at:

`/content/drive/MyDrive/DNS_Hawk/modelD1/wildcard_capture_assignment.csv`

The resulting partition was:

Wildcard partition	Number of PCAPs	Purpose
Wildcard training	8	Added as benign examples during D1/D2/D3 training
Reserved wildcard final test	5	Kept out of model fitting and evaluated as benign traffic

The split was made reproducibly using a randomized selection of five captures for final testing (`random_state=42`); the other eight were designated for training.

The actual assignment used the `d1_role` column:

- `wildcard_train` — 8 captures
- `wildcard_final_test` — 5 captures

This distinction matters because the wildcard dataset's initial intended role was false-positive testing. For the D-series experiments, we deliberately used eight captures for training and reserved five for final testing. We did not treat all 13 as unseen test traffic.

The reserved five captures were later loaded from their per-capture feature CSVs. Each row from these captures was assigned the benign label `0` and category `wildcard`.

7. Model D1 — wildcard-aware model with minimum 10 queries

D1 was the first wildcard-aware model.

7.1 Training data construction

D1 reused Model A's training/validation split and its `query_count >= 10` threshold.

The training set was built from:

1. Model A's sampled training windows: 1,609 benign and 1,070 tunnel.
2. Feature windows from the eight wildcard training captures, labelled benign.

The wildcard training captures contributed 328 eligible windows at this threshold.

Training component	Benign	Tunnel
Model A training data	1,609	1,070
Added wildcard windows	328	0
D1 final training data	1,937	1,070

Total D1 training size: 3,007 windows.

D1 kept Model A's validation set unchanged:

Validation component	Benign	Tunnel	Total

Normal/tunnel validation	454	86	540

The validation set did not include the reserved five wildcard test captures. The wildcard examples were added to training so the model could learn that wildcard DNS can be benign.

7.2 Training configuration

D1 used the same 17 features and the same XGBoost hyperparameters described earlier. Its final model was saved at:

`/content/drive/MyDrive/DNS_Hawk/modelD1/models/xgboost_dns_tunnel_d1.json`

The D1 validation metrics at threshold `0.5` were perfect on this particular validation split: 540/540 correctly classified, with 454 true negatives and 86 true positives.

That result is encouraging but should not be interpreted as proof of perfect detection in general. The validation data still consisted of the original normal and known-tunnel categories; the reserved wildcard captures were evaluated separately.

7.3 D1 final held-out test

D1's final test consisted of:

- Unknown-tunnel windows: 761
- Cross-endpoint windows: 394
- Reserved wildcard windows: 252
- Total: 1,407 windows

All rows in this test had `query_count >= 10`.

The results at threshold `0.5` were:

Metric	**D1 final test**
Accuracy	98.58%
Precision	100%
Recall	98.27%
F1-score	99.13%
ROC-AUC	99.43%

Average precision	99.89%
TN	252
FP	0
FN	20
TP	1,135

Category-level results:

Category	Windows	Correctly detected tunnel windows	Missed tunnel windows / false positives
`unkownTunnel`	761	749	12 missed
`crossEndPoint`	394	386	8 missed
`wildcard`	252	—	0 false positives

The wildcard category contains benign examples, so its relevant result is zero false positives rather than tunnel recall.

D1 artifacts were saved under `/content/drive/MyDrive/DNS_Hawk/modelD1/test/`, including prediction probabilities, classification reports, overall metrics, category metrics, confusion matrices, ROC/PR curves, and feature importance.

8. Model D2 — wildcard-aware model with minimum 5 queries

D2 tested whether a lower minimum query count could retain strong performance after wildcard-aware training.

8.1 Training data construction

D2 reused Model B's train/validation split, applying the `query_count >= 5` filter. It added benign feature windows from the same eight wildcard training captures.

The training workflow used Model B's existing training dataset and the same 17 features and XGBoost configuration as D1.

The benign-to-tunnel ratio was controlled using the training sampling strategy; wildcard windows were included as benign training examples. The reserved five wildcard test captures remained separate.

The exact D2 final training and validation class counts are not available in the retained results summary, so I will not invent those counts. The original Model B partition sizes, before the D2 wildcard addition, were 4,057 training rows and 967 validation rows.

D2 was saved at:

`/content/drive/MyDrive/DNS_Hawk/modelD2/models/xgboost_dns_tunnel_d2.json`

8.2 D2 final held-out test

D2's individual final test used:

- Unknown-tunnel windows: 770
- Cross-endpoint windows: 409
- Reserved wildcard windows: 254
- Total: 1,433 windows

All rows met `query_count >= 5`.

Metric	D2 final test
Accuracy	96.86%
Precision	100%
Recall	96.18%
F1-score	98.05%
ROC-AUC	97.97%
Average precision	99.63%
TN	254

FP	0
FN	45
TP	1,134

Category-level results:

Category	Windows	Correctly detected tunnel windows	Missed tunnel windows / false positives
`unkownTunnel`	770	746	24 missed
`crossEndPoint`	409	388	21 missed
`wildcard`	254	—	0 false positives

D2's results were good, but it missed more tunnel windows than D1 in their respective individual tests. Those individual tests had different numbers of eligible windows, however, so a direct comparison required one shared evaluation set.

Artifacts were saved under `/content/drive/MyDrive/DNS_Hawk/modelD2/test/`.

9. Model D3 — wildcard-aware model with minimum 2 queries

D3 evaluated the lowest query-count threshold.

9.1 Training data construction

D3 reused Model C's training/validation split, filtered to `query_count >= 2`, and added benign examples from the same eight wildcard training captures.

Its strategy retained the maximum 3:1 benign-to-tunnel training ratio. It retained the eligible tunnel rows, capped the combined benign rows at three times the tunnel count, and sampled benign rows as needed.

An important methodological detail: Model C's saved training dataset was already downsampled. D3 started from that sampled training set, then added wildcard examples and reapplied the benign cap. Therefore, the wildcard examples replaced some of the existing sampled benign rows rather than being added to every eligible benign row from the original extraction.

That distinction should be documented accurately.

D3 was saved at:

`/content/drive/MyDrive/DNS_Hawk/modelD3/models/xgboost_dns_tunnel_d3.json`

9.2 D3 individual final held-out test

The original D3 final test used:

- Unknown-tunnel windows: 795
- Cross-endpoint windows: 421
- Reserved wildcard windows: 267
- Total: 1,483 windows

All rows met `query_count >= 2`.

Metric	D3 individual final test
Accuracy	94.00%
Precision	99.74%
Recall	92.93%
F1-score	96.21%
ROC-AUC	95.89%
Average precision	99.23%
TN	264
FP	3
FN	86

TP	1,130

Category-level results:

Category	Windows	Correctly detected tunnel windows	Missed tunnel windows / false positives
`unkownTunnel`	795	744	51 missed
`crossEndPoint`	421	386	35 missed
`wildcard`	267	—	3 false positives

D3 detected fewer tunnel windows and produced three wildcard false positives. It nevertheless evaluated more low-activity windows, which was its purpose.

Artifacts were saved under `/content/drive/MyDrive/DNS_Hawk/modelD3/test/`.

10. The controlled final comparison: D1 vs D2 vs D3

The individual final tests above were constructed using different minimum query-count thresholds. Consequently, their total window counts differed. We then ran a controlled comparison to make the model comparison fairer.

10.1 How the shared test set was built

The shared test set was constructed from two sources:

1. `heldout_common_ge2.csv` — the held-out non-wildcard categories, `unkownTunnel` and `crossEndPoint`.
2. The five reserved wildcard PCAPs — loaded from their hashed per-capture feature CSVs and assigned benign label `0`.

The common held-out dataset originally had 1,842 rows and 26 columns. It contained the 17 selected model features, two excluded response-derived features, the label, and metadata.

The evaluation code selected the 17 features and the label/category fields, removed wildcard rows from the common dataset to avoid duplicating wildcard traffic, and then appended the reserved wildcard capture windows.

The combined shared test set contained 1,484 rows before query-count filtering:

Category	Rows before filtering
`unkownTunnel`	795
`crossEndPoint`	421
`wildcard`	268
Total	1,484

The two controlled tests were then generated from that same combined table.

10.2 Controlled Test A — minimum 10 queries

All three models were evaluated on the same 1,407 windows, using threshold `0.5`.

Metric	D1	D2	D3
Accuracy	98.58%	98.29%	97.94%
Precision	100.00%	100.00%	99.73%
Recall	98.27%	97.92%	97.75%
F1-score	99.13%	98.95%	98.73%
ROC-AUC	99.43%	99.33%	99.22%
Average precision	99.89%	99.88%	99.86%

TN	252	252	249
FP	0	0	3
FN	20	24	26
TP	1,135	1,131	1,129

The category breakdown was:

Category	Windows	D1 missed / FP	D2 missed / FP	D3 missed / FP
`unkownTunnel`	761	12 missed	16 missed	18 missed
`crossEndPoint`	394	8 missed	8 missed	8 missed
`wildcard`	252	0 FP	0 FP	3 FP

10.3 Controlled Test B — minimum 5 queries

All three models were evaluated on the same 1,433 windows, again using threshold `0.5`.

Metric	D1	D2	D3
Accuracy	97.35%	96.86%	96.37%
Precision	100.00%	100.00%	99.74%
Recall	96.78%	96.18%	95.84%
F1-score	98.36%	98.05%	97.75%
ROC-AUC	98.61%	97.97%	98.07%

Average precision	99.74%	99.63%	99.65%
TN	254	254	251
FP	0	0	3
FN	38	45	49
TP	1,141	1,134	1,130

The category breakdown was:

Category	Windows	D1 missed / FP	D2 missed / FP	D3 missed / FP
`unkownTunnel`	770	17 missed	24 missed	26 missed
`crossEndPoint`	409	21 missed	21 missed	23 missed
`wildcard`	254	0 FP	0 FP	3 FP

The category counts and overall confusion matrices agree: the test sets contain 1,155 tunnel windows in Test A and 1,179 tunnel windows in Test B. The remaining windows are benign wildcard traffic.

10.4 Interpretation

D1 was the strongest model in both controlled comparisons:

- It missed fewer tunneling windows than D2 and D3.
- It produced zero false positives on the reserved wildcard windows in both tests.
- It achieved the highest overall F1-score and ROC-AUC in both tests.
- D2 also achieved zero wildcard false positives, but missed more tunneling windows.
- D3 missed the most tunneling windows and generated three wildcard false positives in each controlled test.

The comparison supports selecting D1 as the provisional model for integration into DNSHawk. It does not establish that D1 will perform identically on unseen networks or arbitrary live DNS traffic.

11. Complete preprocessing and splitting summary

This is the compact version of the full experimental workflow.

Stage 1 — Raw data

122 PCAPs

Five categories; nine parent captures excluded where segmented PCAPs were present.

Stage 2 — Extraction

898,737 feature rows

TShark parsing, DNS event construction, 60-second aggregation and 19 features.

Stage 3 — Model features

17 query-only features

Excluded NXDOMAIN rate and average response length because the live response fields were unreliable.

Stage 4 — Baseline split

Group-aware train/validation split

StratifiedGroupKFold, five folds, first split, fixed random seed 42. Normal and known-tunnel categories used for baseline fitting and validation.

Stage 5 — Imbalance handling

Maximum 3:1 benign-to-tunnel ratio

Benign training rows downsampled; validation rows not downsampled.

Stage 6 — Wildcard-aware models

D1 / D2 / D3

Added windows from eight wildcard training captures. Kept five wildcard captures reserved for final testing.

Stage 7 — Controlled evaluation

1,407 windows at ≥10; 1,433 at ≥5

Same test windows across all three models within each threshold; probability threshold 0.5.

12. What was actually held out, and what still needs verification?

The intended methodology was to separate captures and capture groups, rather than randomly split individual windows. That is the correct general approach for reducing leakage from

correlated traffic.

However, before treating the numbers as final research results, there are three checks worth completing.

1. Verify capture-group isolation for every model. Confirm that the train, validation, and final test sets have no overlapping PCAP capture groups. In particular, inspect the source PCAP paths and group IDs rather than relying only on the row labels.

2. Verify the wildcard split. Confirm that the eight `wildcard_train` captures were used for model fitting and the five `wildcard_final_test` captures were never used for fitting, feature selection, threshold selection, or other model choices. The controlled evaluation code uses the `d1_role` column to select the reserved five.

3. Keep the test set untouched during model selection. Since the held-out test results have now been examined, they should be treated as a reported evaluation set rather than repeatedly used to choose model settings. If you tune the decision threshold based on these test predictions, the resulting figures become development results; a separate untouched test set would be preferable for a final independent estimate.

Final conclusion

The experiment progressed from a heavily imbalanced dataset of 898,737 feature windows to group-aware baseline models, then to wildcard-aware D1/D2/D3 models. The controlled comparison favors D1, which used a minimum query count of 10 and achieved 98.58% accuracy, 98.27% recall, a 99.13% F1-score, and zero false positives on the reserved wildcard windows in the controlled `query_count >= 10` evaluation.

These are results on the constructed PCAP test set, not a guarantee of live-network performance. The next stage should be threshold analysis and live-pipeline validation, with the 17-feature schema kept consistent between training and inference.